

Q-1:- Brief History & paradigms, application of AI. Symbolic AI, Subsymbolic AI, What is AI, What is ML, The rise of data driven AI.

2) Intelligent Agents; rational agents agent environments; PEAS, frameworks Types of agents.

Artificial Intelligence:- Artificial means "man-made", and Intelligence defines "Thinking power", hence "AI means a man made thinking power."

Machine Learning:- It is a subset of AI. which enables systems to learn from data & improve their performance on a specific task over time without being explicitly programmed.
* They identify patterns & make predictions or decisions based on the data.

ML Types

- ① Supervised Learning.
- ② Unsupervised Learning.
- ③ Reinforcement Learning.

→ Alan Turing introduced the Turing test & Computing, machinery & learning in 1950s.

Applications of AI

- 1) Game playing.
- 2) Medical Diagnosis, Autonomous Control.
- 3) Autonomous Planning & Scheduling.
- 4) Expert Systems.
- 5) Robotics.
- 6) Natural Language Processing.
- 7) Computer vision.
- 8) E-commerce.
- 9) Agriculture.
- 10) Finance.

AI in healthcare

- Diagnostics
- Personalized Medicine
- Robotic Surgery.
- Drug Discovery.

AI in Smart Homes & Cities

- Home Automation
- Smart Grids
- Urban Planning.

AI in Education

- Personalized Learning.
- Tutoring Systems.
- Administrative Tasks.

AI in Transport

- Autonomous Vehicles
- Traffic Management
- Logistics & Supply Chain

AI in Agriculture

- Precision Farming
- Crop Monitoring
- Yield Prediction.

AI in Finance

- Fraud Detection.
- Algorithmic Trading.
- Risk Management.
- Customer Service.

Symbolic AI

- AI based on explicit knowledge representation using symbols, rules, & logic. It focuses on representing human knowledge in a structured, human-readable form.

* Systems use logical reasoning & predefined rules to manipulate these symbols and draw conclusions.

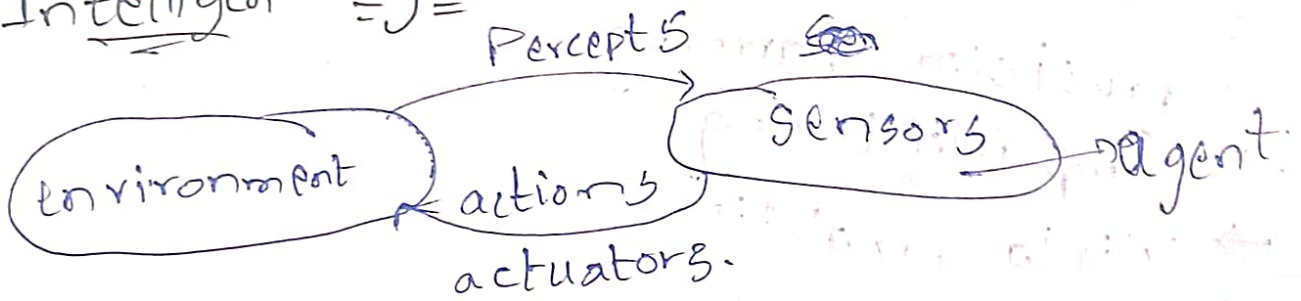
Sub-Symbolic AI

→ AI that learns from data without explicit symbolic rules.

- * Systems learn patterns & relationships directly from large datasets through numerical methods & statistical models. Knowledge is distributed across the network's connections.

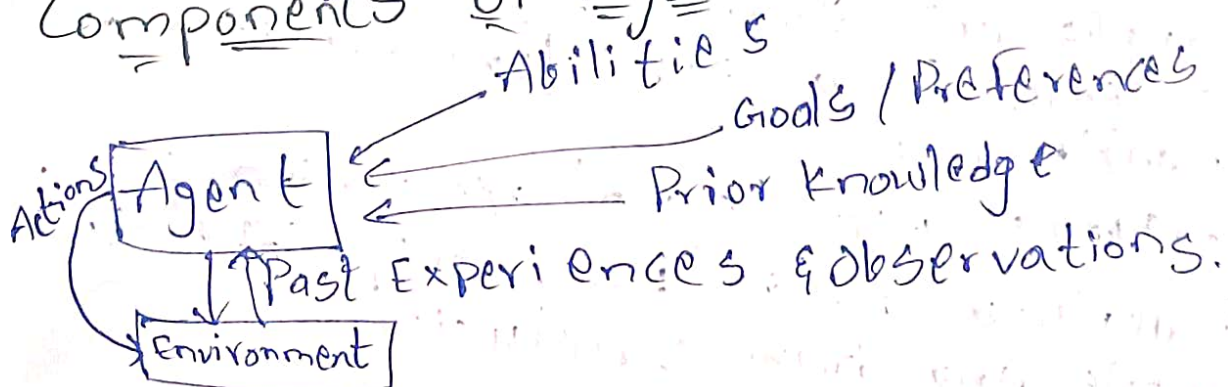
22/07/25

Intelligent Agent



- * Agent is an element in a system or environment that can trigger action.
- * Agent makes decision based on how they perceive their surroundings.
- * Sensor has the perception capability.
- * Actuators for machines are motors.

Components of Agent



Agents Example

→ A software agent → keystrokes, file content
received network package as sensors.

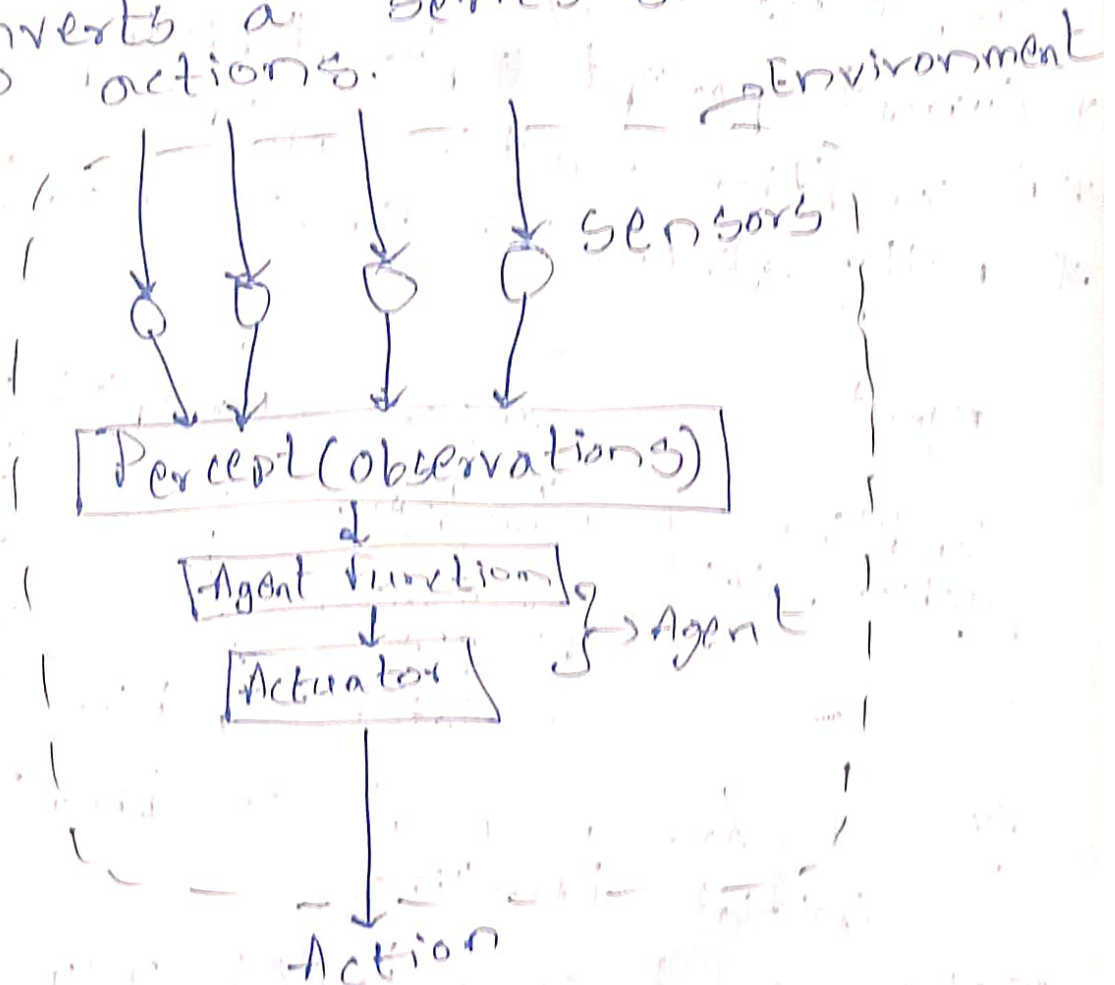
* Displays on the screen, files, sent
network packets as actuators.

→ Robotic agent → sensors = cameras &
infrared range finders
Actuators → motors.

Agent Function

environment → sensors → agent function
→ actuators → environment

* It is a mathematical formula
converts a series of observations
into actions.



Rational Agent

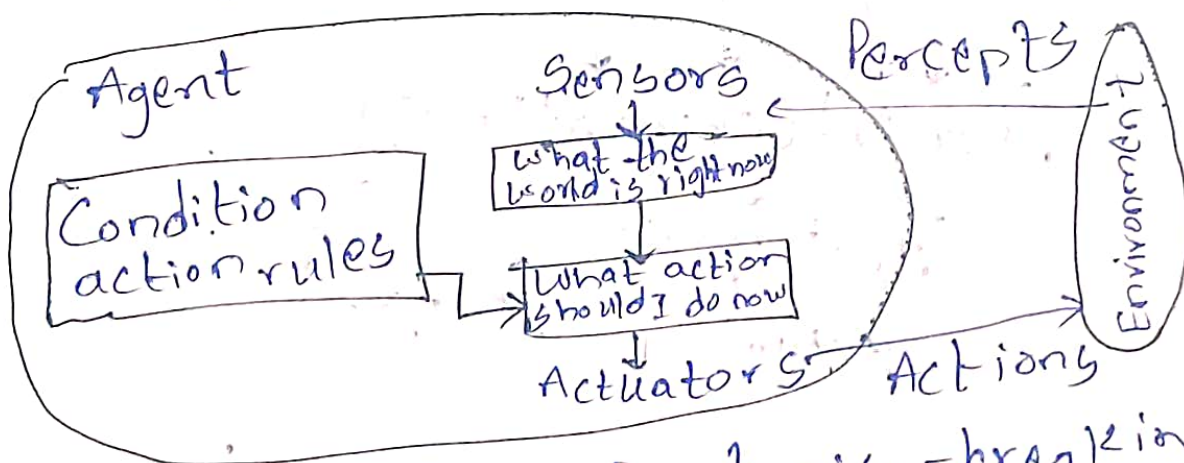
- This makes the best choice in any circumstance. (Basis of rationality)
- Through performance measure, we can make the best choice.

Types of Agent

- Simple reflex agents
- Model-based reflex agents
- Goal-based
- Utility-based.

Simple reflex Agents

- * Current percept ^{ignores} percept history
- * A condition-action rule
- * fully observable Environment.



Ex: - if car in front is breaking then initiate breaking.

- * This agent concentrates only on the current input, ignoring history or consequences.

~~Ex~~ This works on
IF < condition > THEN < action >
Ex:- IF "traffic light is red" → THEN "stop".

IF "floor is dirty" → THEN "vacuum".

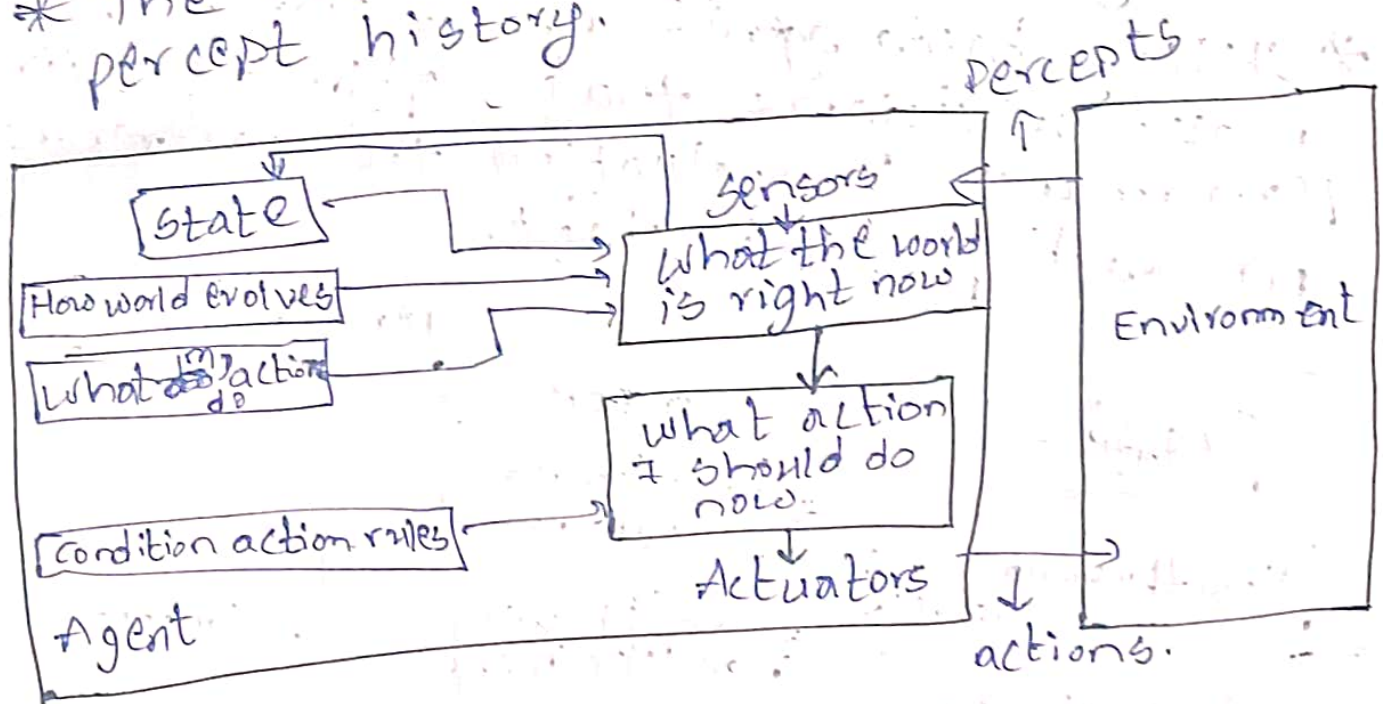
23/07/25

Model based reflex Agents

* Partially observable.

* The agent tracks its internal state which is adjusted based on each new percept.

* The internal state depends on the percept history.

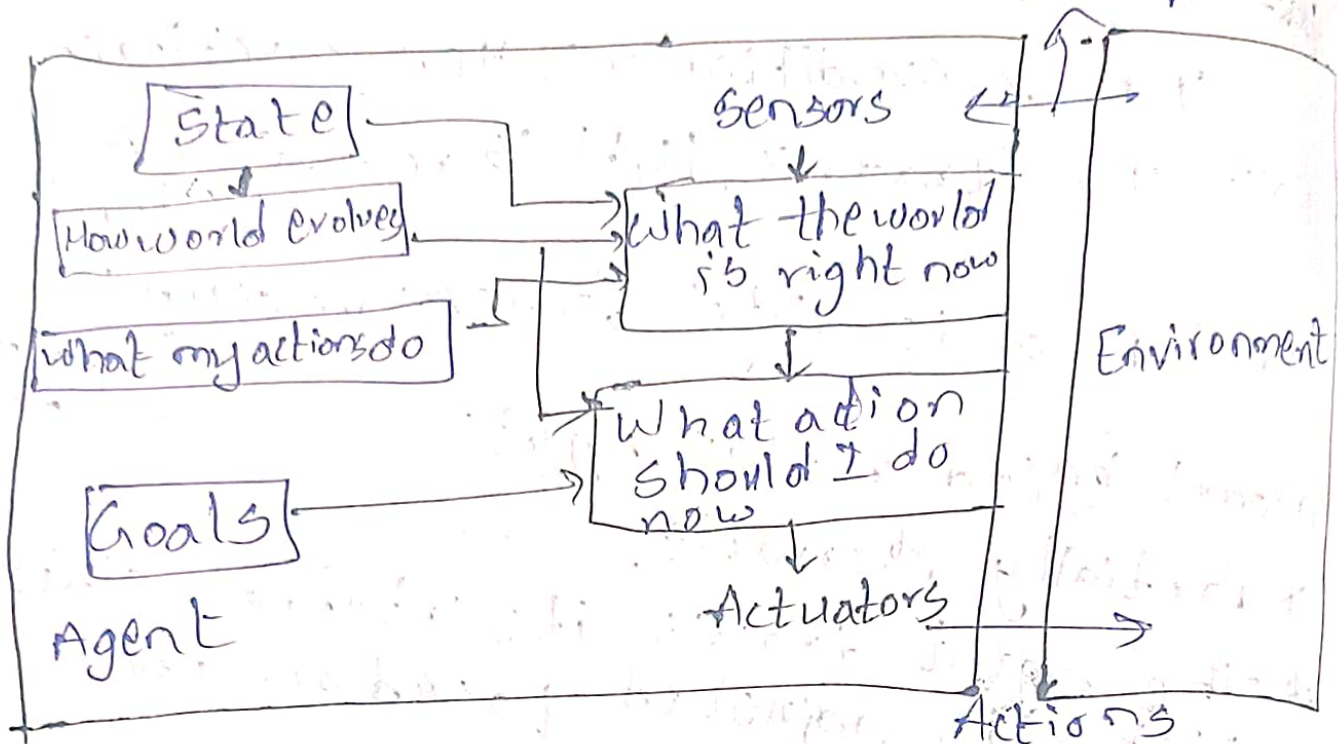


Ex:- Yomo car

Goal-Based Agents

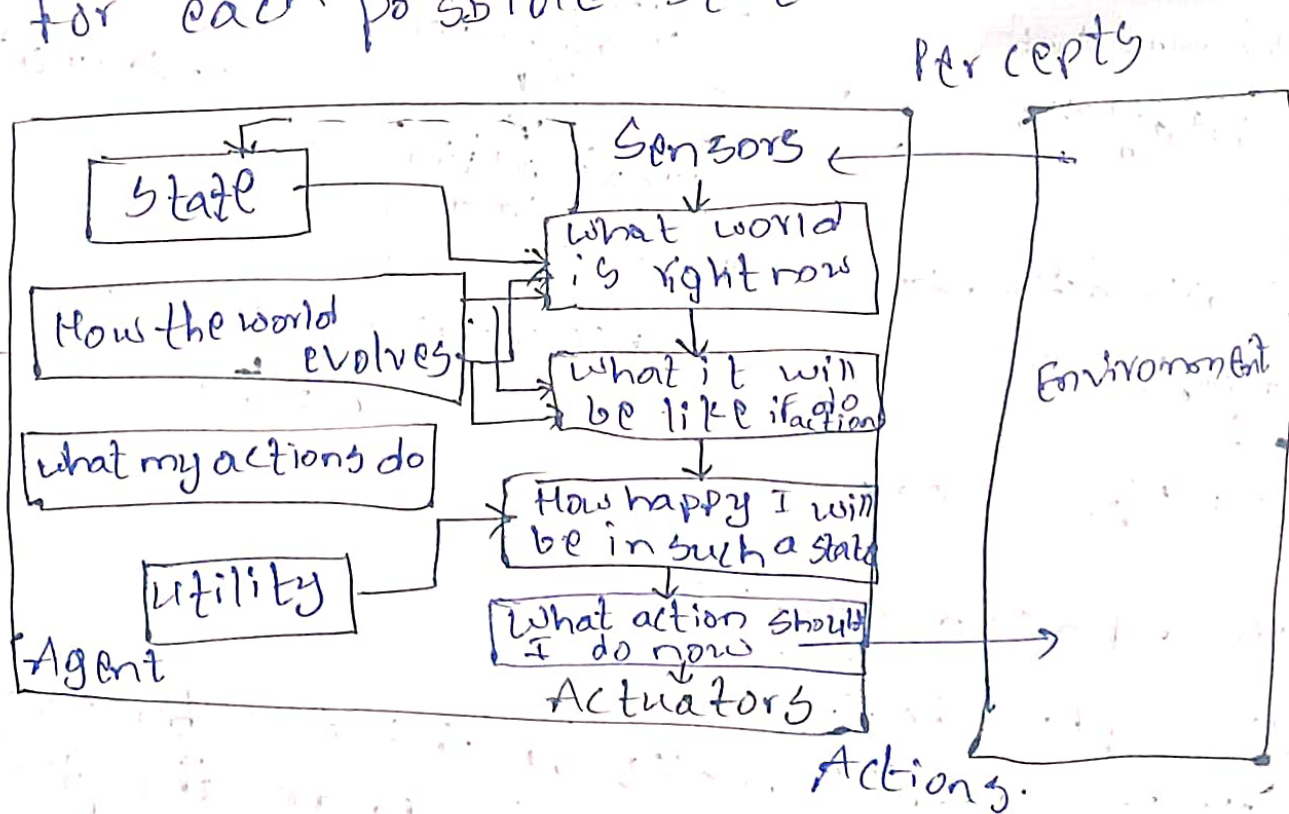
Ex:- A plus-Robo

* These agents make decisions based on their current distance from the goal and every action the agent aims to reduce the distance from goal.

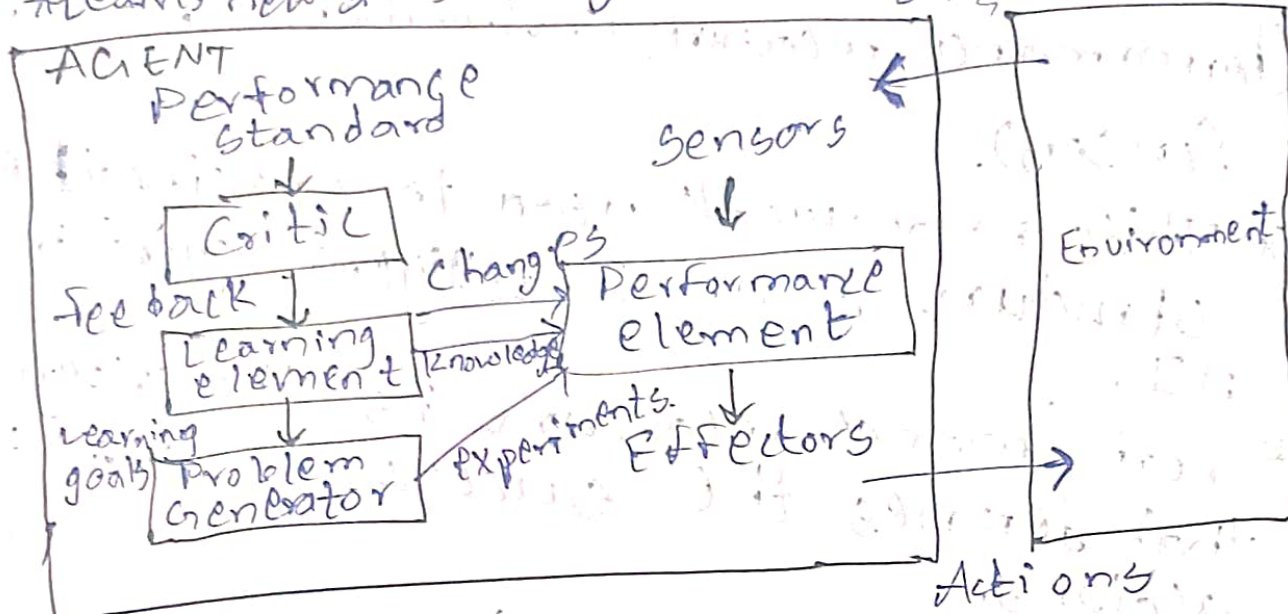


Utility Based Agents

* Utility based agents are designed to make decisions that optimize their performance by evaluating the preferences for each possible state.



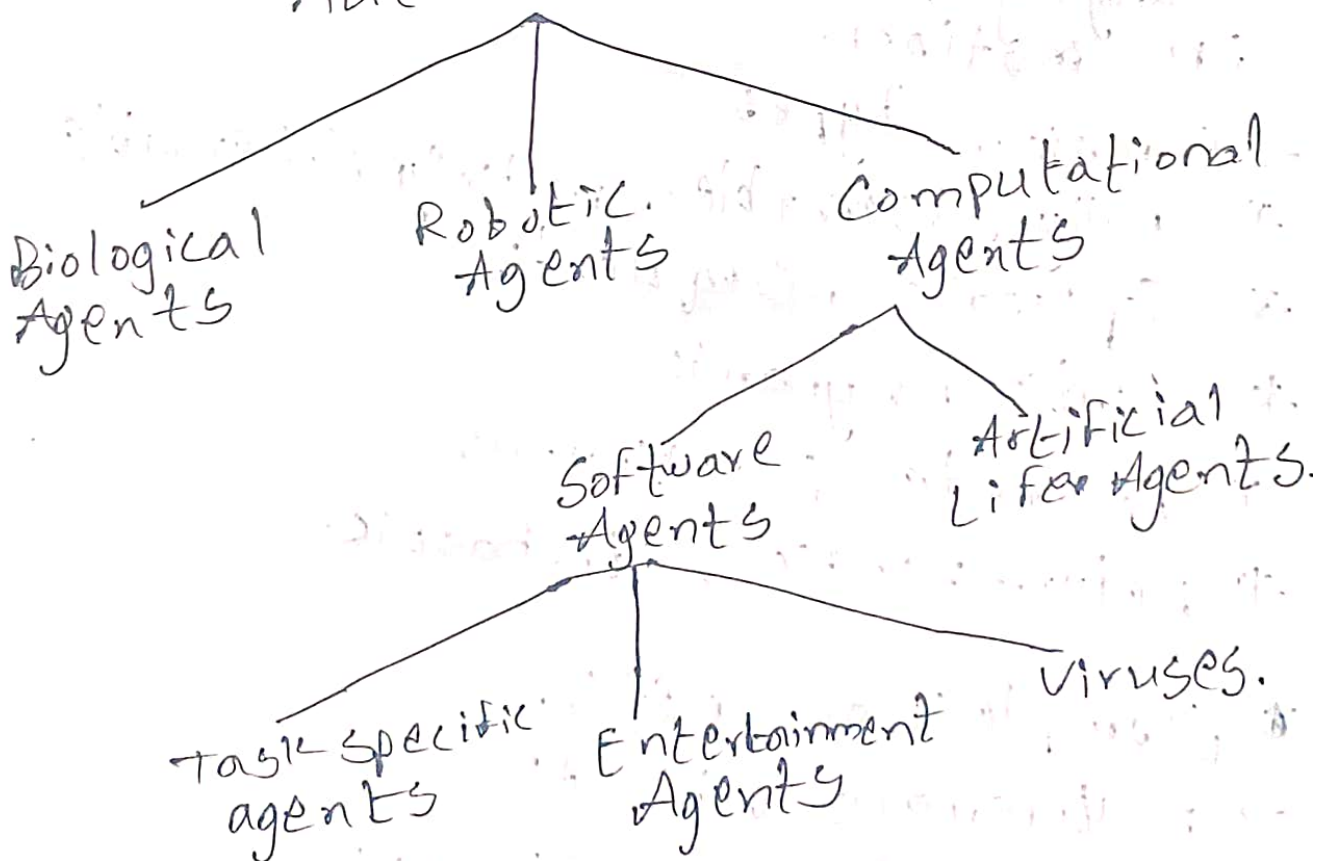
Learning Agent
 learns new skills through observations ^{Percepts} ~~Actions~~ ^{acts}.



→ Multi Agent System
 → Hierarchical Agents.

Agent Classification

Autonomous Agents



PEAS

(Performance measure, Environment, Actuator, Sensors).

* This framework used to specify the structure of an Intelligent agent in AI.

* Performance measure is the criteria that defines the success of agent's actions.

Ex:- In a self driving car (safety, efficiency & comfort).

* Environment is the surroundings of an agent at every instant. It keeps changing with time if the agent is set in motion.

→ 5 major types.

* Fully observable & Partially observable

* Episodic & Sequential

* Static & Dynamic

* Discrete & continuous

* Deterministic & Stochastic.

* Examples of PEAS.

→ Autonomous vehicles

→ Robotic Vacuum Cleaners.

→ Game - Playing AI.

States, transitions, costs, search spaces
applications of Classical Search: Uninformed
(BFS, DFS, UCS, IDS); Informed (Greedy, A*)

29/07/25

Key Components of Search Problem

1) initial state

2) actions

3) transition model

4) goal test

5) path cost

6) search space

Problem formulation
→ process of defining
a problem in a structured & solvable way.

1) Initial state

* Represents the starting point of the agent in the environment.

Ex:- In navigation task

2) Actions (Operators)

* They are set of all legal moves or operations that an agent can perform in a particular state.

Ex:- In a maze-solving problem, actions may include "move forward", "turn left", or "turn right".

3) Transition model

* When action is applied to a state transition model happens.

Ex:- "If the agent is in state S and performs action A , what will the resulting state S' be?"

$\therefore \text{Result}(S, A) = S'$.

4) Goal Test

* This determines whether a given state is a goal state.

Ex:- In 8-puzzle problem, the goal test checks whether the tiles are in specific order or not.

5) Path cost

* It is the cumulative cost of all actions taken along the way.

* Objective of many AI search algorithms is to find the one with the least path cost.

6) Search space

* This refers to the entire set of possible states the agent can reach from the initial state by applying action.

* Visualized as a search tree or graph, where nodes represent states & edges represent actions.

Steps in Problem Formulation

→ Structuring a real world issue into a well-defined computational problem that an AI system can solve. This requires defining the problem's scope, specifying possible actions, & setting constraints to guide the AI toward an optimal solution.

Step 1:- Define problem statement.

Ex:- AI based package delivery system.
= → Warehouse A to Customer.

Step 2:- Establish the initial state to Goal state.

Initial State:- Warehouse A

Goal State:- Customer B

Step 3:- Determine available actions & State transition model.

* Move forward, turn left or right or stop.

Step 4:- Define constraints & path cost.

Traffic rules, package, delivery deadlines, shortest path, lowest fuel consumption.

→ Class Delivery Agent:

```
def __init__(self):
```

```
    self.location = "Warehouse A"
```

```
    self.destination = "Destination B"
```

```
    self.path = [] (stores the path  
                    taken by the agent)
```

```
    def get_current_state(self):
```

```
        return f"Package is currently at self.  
        location location {self.location}"
```

→ Class Actions:

@ static method

```
def move_forward(agent):
```

```
    agent.path.append("move forward")
```

```
    return "moving forward"
```

@ static method

```
def turn_left(agent):
```

```
    agent.path.append("Turn left")
```

```
    return "Turn left"
```

@ static method

```
def turn_right(agent):
```

```
    agent.path.append("Turn right")
```

```
    return "Turn right"
```

@ static method

```
def stop(agent):
```

```
    agent.path.append("stop")
```

```
    return "Stopping"
```

→ Class Transition Model:

@static method:

```
def update_state(agent.action):
```

```
    if action == "Move Forward":
```

```
        agent.location = "Next Location":
```

```
    elif action == "Turn left":
```

```
        agent.location = "Left Turn Location"
```

```
    elif action == "Turn right":
```

```
        agent.location = "Right Turn Location"
```

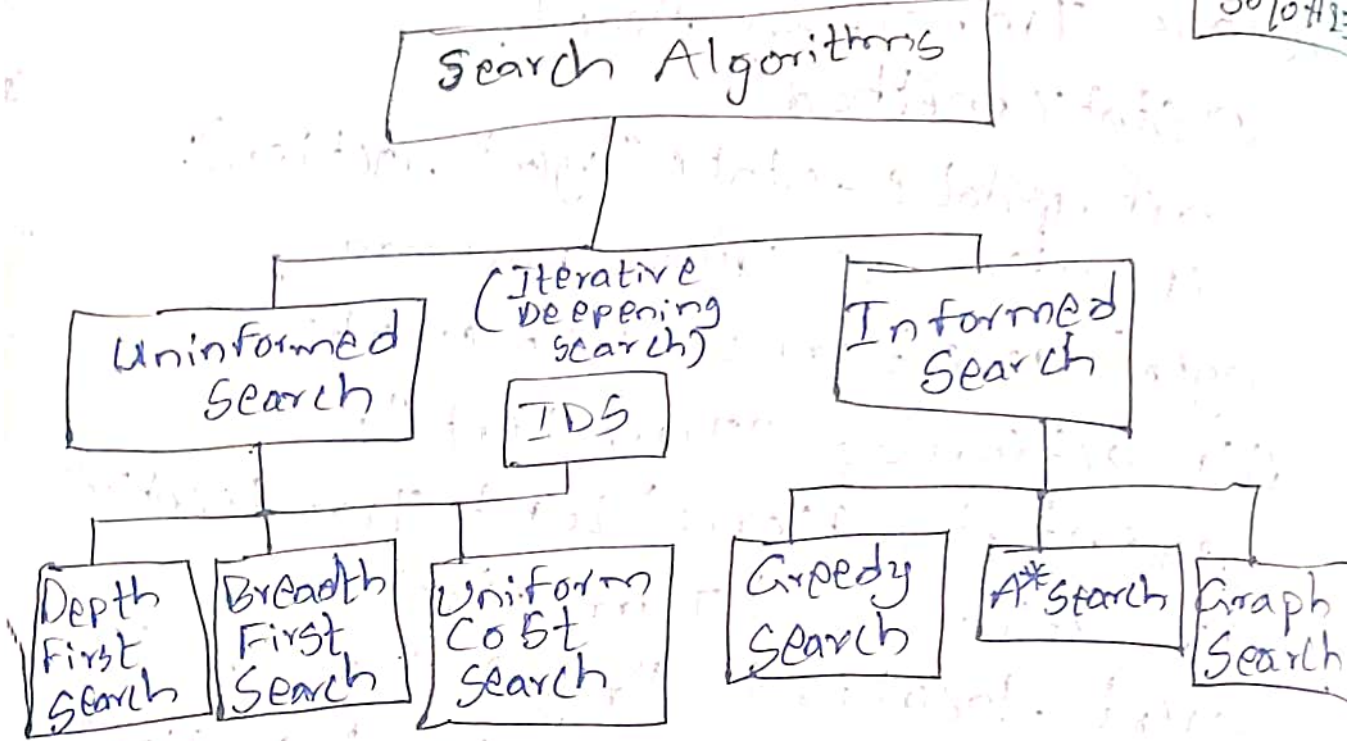
```
    return f"New location: {agent.location}"
```

→ def check_goal_state(agent):

```
    return agent.location == agent.destination
```

→ def path_cost_function(distance, fuel_usage, traffic_delay):

```
    return distance * fuel_usage + traffic_delay
```

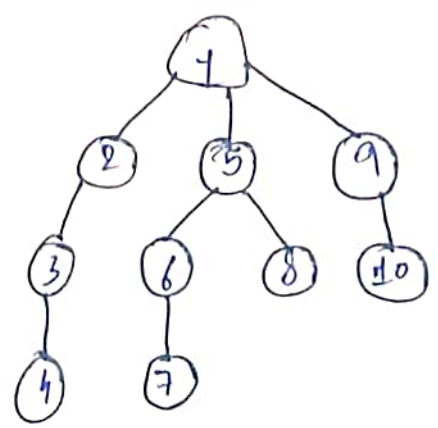


Uninformed Search Algorithms

- * The search algorithms have no additional information on the goal node other than the one provided in the problem definition
- * This is also called as Blind Search.
- * ~~These go~~

Depth First Search (DFS)

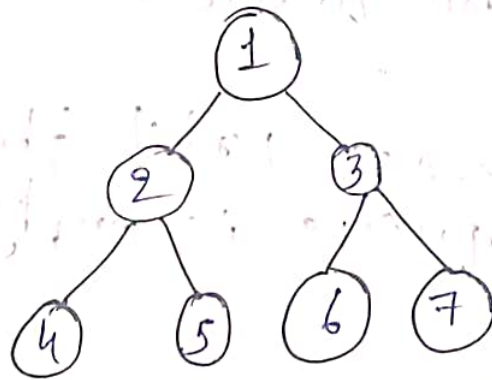
- * Performs the depth-wise search to find the goal node.



Algorithm

- 1) Initialize an empty stack and push the root node onto it.
- 2) While the stack is not empty:
 - (a) Pop the top node.
 - (b) If this node is the goal, return the solution.
 - (c) Else, push all unvisited neighboring nodes onto the stack.
- 3) Repeat the process until the goal is found or the stack is empty.

Ex:-



D.F.S $\Rightarrow$ 1 2 4 5 3 6 7

Advantages:-

- $\rightarrow$ Memory efficiency
- $\rightarrow$ Finds a solution quickly
- $\rightarrow$ Easy to implement
- $\rightarrow$ Can be used for certain types of problems.

Disadvantages

- $\rightarrow$ Can get stuck in infinite loops
- $\rightarrow$ May not find the optimal solution
- $\rightarrow$ Can take long time.

Breadth First Search (BFS)

* Level-by-Level Search is performed.

Algorithm

1) Initialize an empty queue & enqueue the root node.

2) While the queue is not empty:

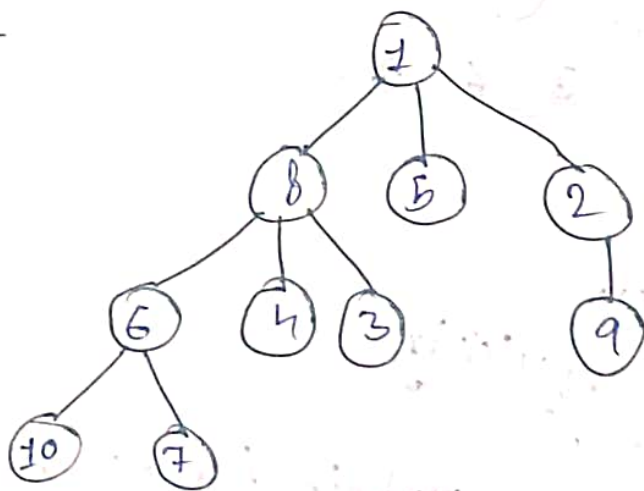
(a) Dequeue the front node

(b) If this node is the goal, return the solution.

(c) Else, enqueue all unvisited neighboring nodes.

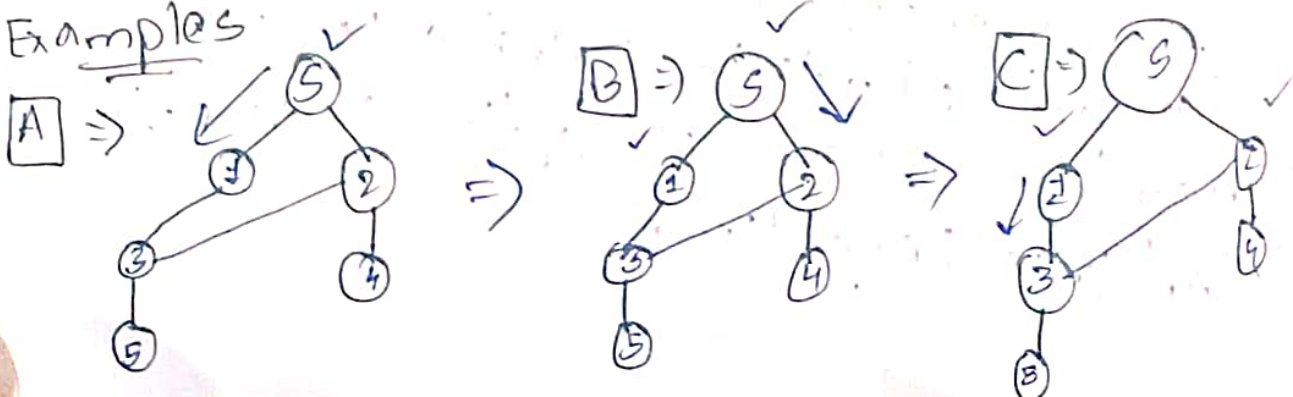
3) Repeat the process until the goal is found or the queue is empty.

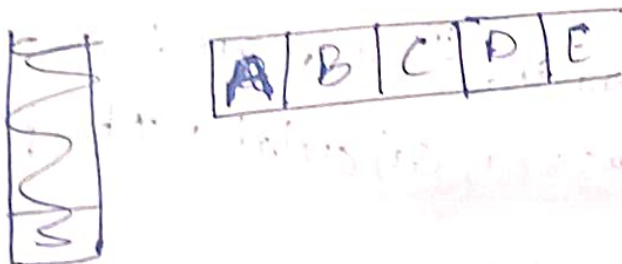
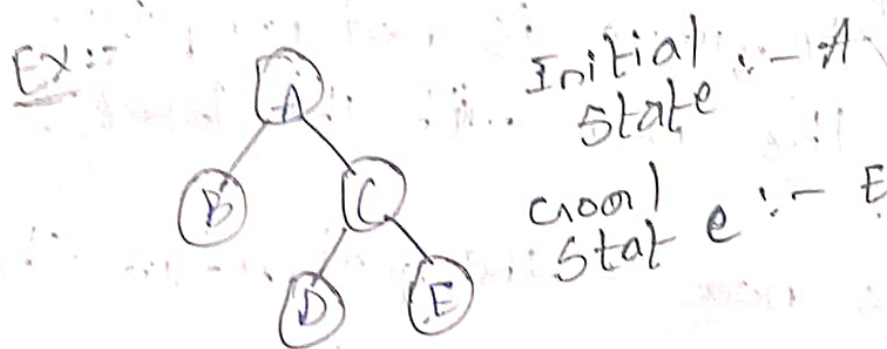
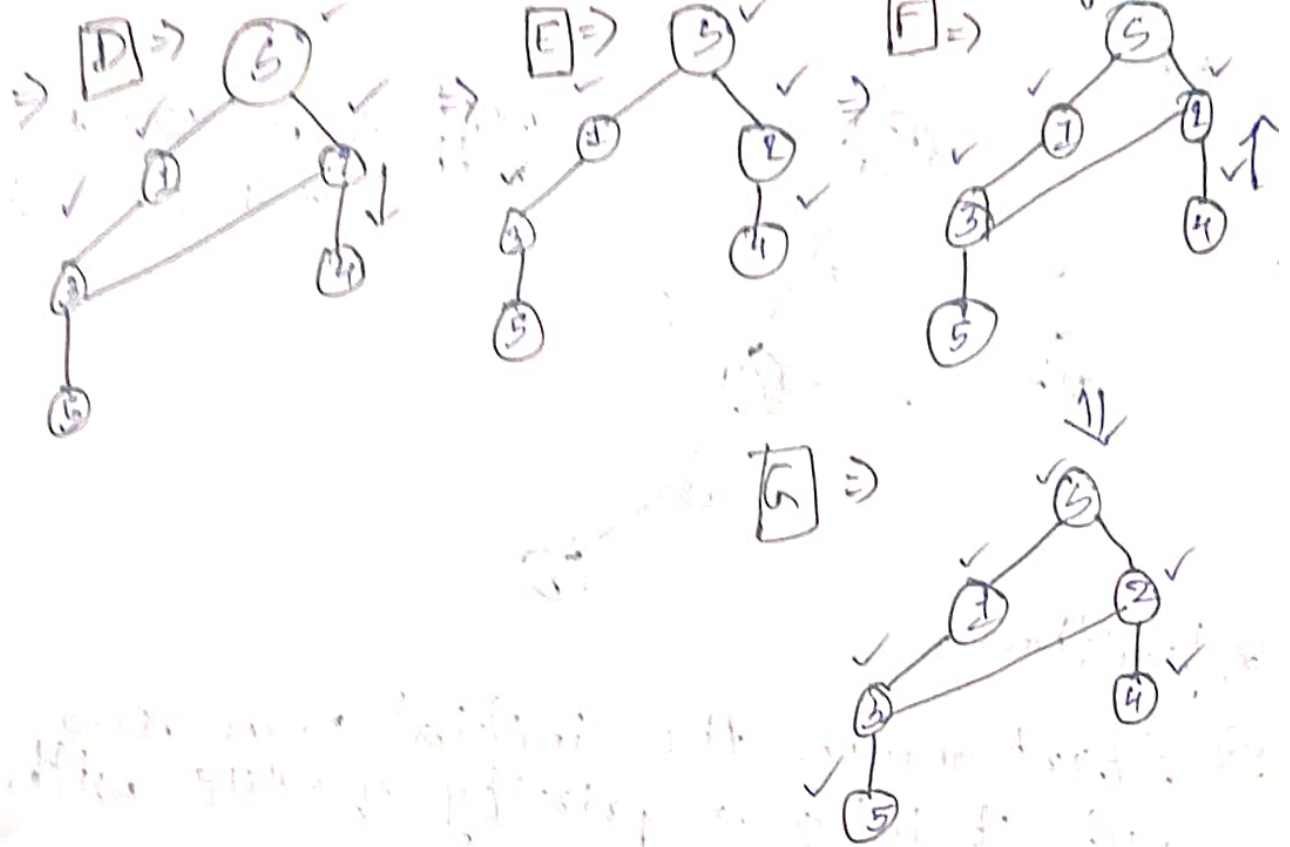
Ex:-



BFS:- 1 8 5 2 6 4 3 9 10 7 ✓

Examples

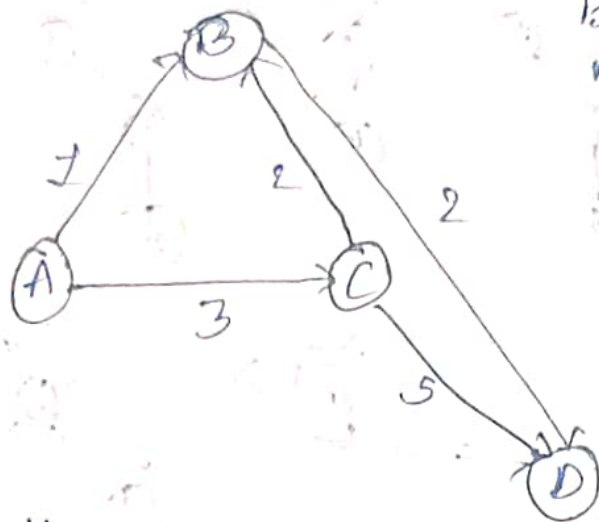




Uniform Cost Search

- * Extension of BFS, by considering path costs, always expanding the least cost node first.
- * Finds the optimal path when all costs are non-negative.
- * Finds the least cost path
- * Slower than BFS
- * This is a graph traversal algorithm

Ex: -

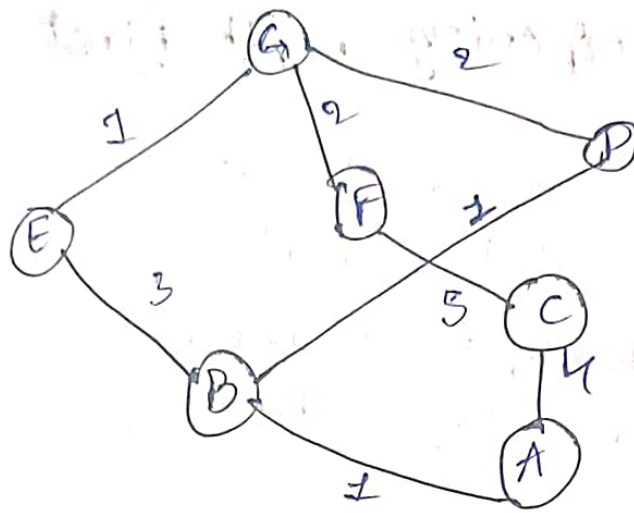


Best Path $\Rightarrow A - B - D$
 $1 + 2 = 3$

Algorithm

- 1) Start from the initial node and put it into a priority queue with cost 0.
- 2) While the queue is not empty:
 - a) Remove the node with the lowest path cost.
 - b) If this node is the goal, return the path.
 - c) Otherwise, expand the node:
 - i) For each successor, calculate the path cost.
 - ii) If the node is new or has a lower cost than previously found, add it to the queue.
- 3) If the queue

Ex:-



- 'A' $\Rightarrow [(B, 1), (C, 4)]$ ① Expand 'A' (cost: 0):
 → Add 'B' (cost: 0+1=1) & 'C' (cost: 0+4=4) to priority queue.
 → Frontier = [(1, B), (4, C)]
 → Visited = {'A': (0, None), 'B': (1, A), 'C': (4, A)}
- 'B' $\Rightarrow [(D, 1), (E, 3)]$
- 'C' $\Rightarrow [(F, 5)]$
- 'D' $\Rightarrow [(G, 2)]$
- 'E' $\Rightarrow [(G, 1)]$ ② Expand 'B' (cost: 1):
 → Add 'D' (cost: 1+1=2) & 'E' (cost: 1+3=4) to the priority queue.
 → Frontier = [(2, D), (4, C), (4, E)]
 → Visited = {'A': (0, None), 'B': (1, A), 'C': (4, A), 'D': (2, B), 'E': (4, B)}
- 'F' $\Rightarrow [(G, 2)]$
- 'G' $\Rightarrow []$

① Initialization

* Create a priority queue (frontier).

② ~~Expansion & Exploration~~

Advantages of UCS

- Optimality Guaranteed
- Complete
- No Heuristics needed.
- Handles Varying Costs.

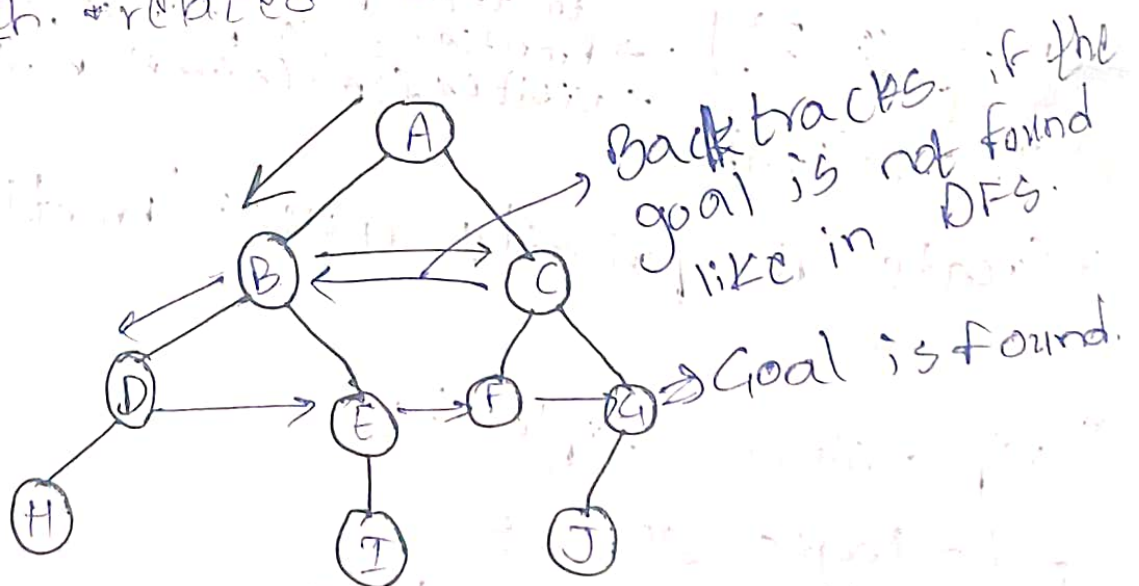
Applications

- Pathfinding
- Puzzle solving
- Game AI
- WebCrawling
- Robot Navigation

Iterative Deepening Depth First Search Algorithm

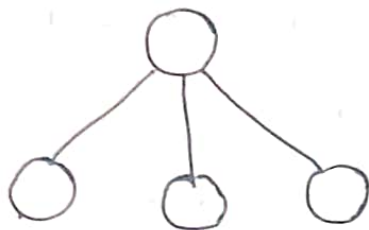
- * Start with a depth limit = 0.
- * Run DFS with that limit.
- * If the goal is found, return the path.
- * If not, increase the depth limit & repeat.

- This is a combination of BFS & DFS.
- Performs DFS repeatedly, increasing the depth limit by one each time.
- First it explores depth 0, then depth 1, depth 2 & so on.
- At each iteration, it performs a depth-related DFS.



Iteration 1 $\Rightarrow A \rightarrow B \rightarrow C$

Iteration 2 $\Rightarrow A \rightarrow B \rightarrow D \rightarrow E \rightarrow F \rightarrow G$



A* Searching Algorithm

→ Combines Dijkstra's algorithm & Greedy Best-First Search.

→ Cost function $\Rightarrow f(n) = g(n) + h(n)$

* $g(n)$: Actual cost from start to current node.

* $h(n)$: Estimated cost from current node to goal (heuristic value).

* $f(n)$: Total estimated cost via node

Algorithm

Initialization: Start node in open list; closed list empty; $g(\text{start}) = 0$; calculate $h(\text{start})$.

Node Selection: Pick node with lowest $f(n)$ from open list.

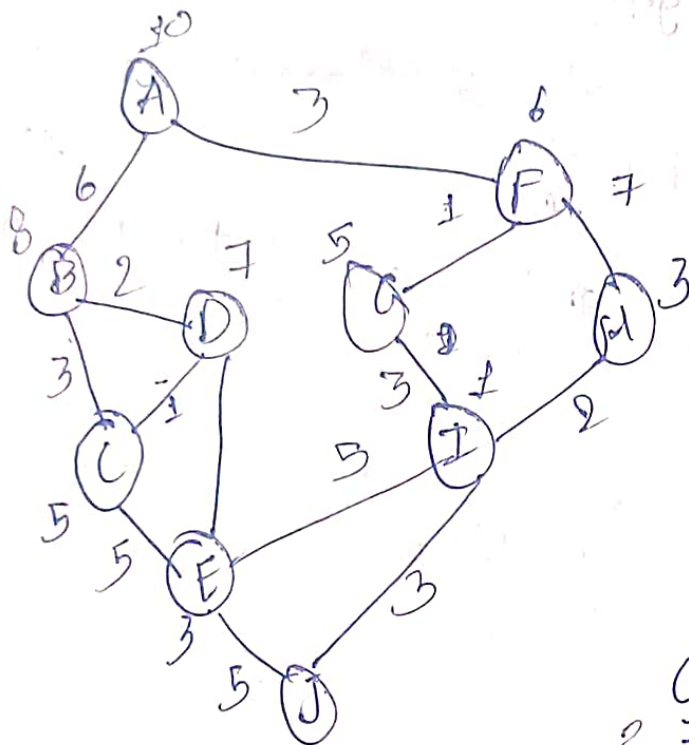
Goal Check: If node = goal, trace path back using parents.

Neighbor Evaluation: Compute tentative costs for neighbors.

Cost Updates: Updates costs & parent if a better path is found.

Move Node: Transfer current node to closed list; repeat.

No path: if open list is empty, no path exists.



$$f(F) = 3 + 6 = 9$$

$$f(B) = 6 + 6 = 12$$

$A \rightarrow F$

$$f(G) = 1 + 5 = 6$$

$$f(H) = 7 + 3 = 10$$

$A \rightarrow F \rightarrow G$

$$f(I) = 1 + 3 = 4$$

$A \rightarrow F \rightarrow G \rightarrow I$

I

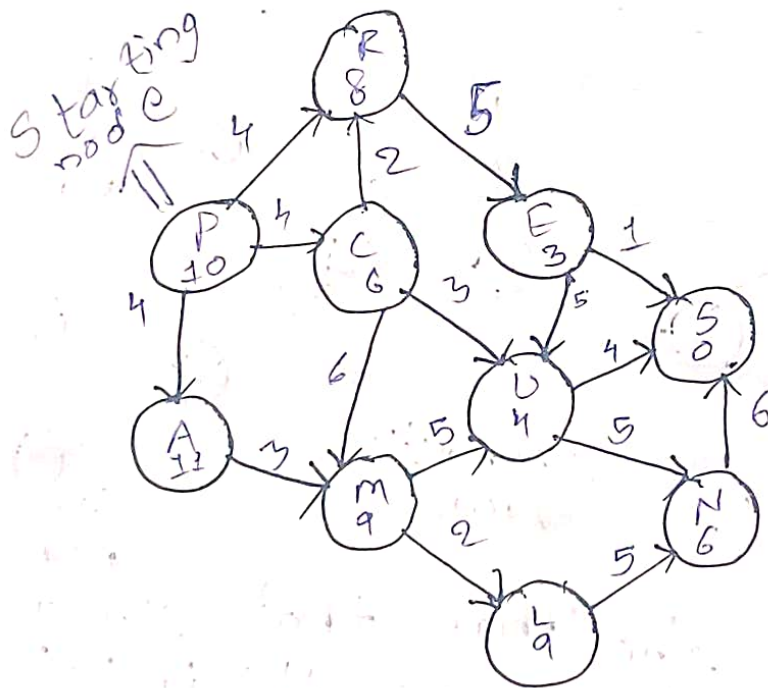
$$f(E) = 5 + 3 = 8$$

$f(J) = 5 + 3 = 8$ This is the goal state.

$A \rightarrow F \rightarrow G \rightarrow I \rightarrow J$

Greedy Search Algorithm

- Expands the node closest to the goal based only on a heuristic function.
- Does not consider the actual cost($g(n)$) from the start node, only the estimated cost to the goal.
- * Cost function $f(n) = h(n)$.



Step 1:-

- From the initial node (P),
 - * To node (A) $\Rightarrow (f(A) = h(A) = 11)$
 - * To node (C) $\Rightarrow (f(C) = h(C) = 6)$
 - * To node (R) $\Rightarrow (f(R) = h(R) = 8)$
- Here the path is $P \rightarrow C$.

Step 2:-

- * To node (R) $\Rightarrow (f(R) = h(R) = 8)$
 - * To node (U) $\Rightarrow (f(U) = h(U) = 4)$
 - * To node (M) $\Rightarrow (f(M) = h(M) = 9)$
- Here the path is $P \rightarrow C \rightarrow U$

Step 3:-

* To node (S) $\Rightarrow (F(N) = h(N) = 0)$

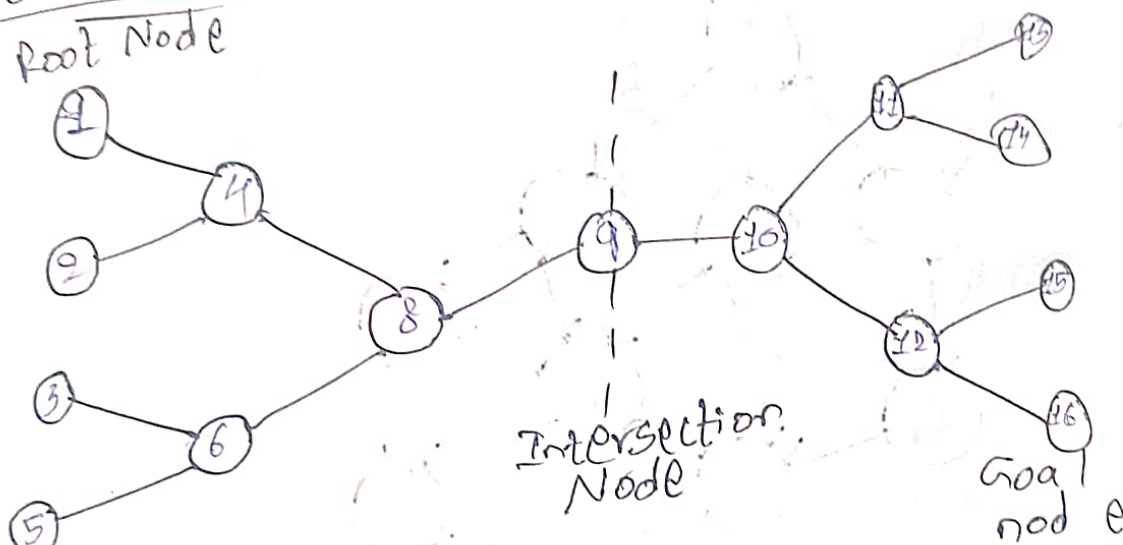
* To node (N) $\Rightarrow (F(S) = h(S) = 6)$

Now we have reached the goal state.

$P \rightarrow C \rightarrow U \rightarrow S$

Bidirectional

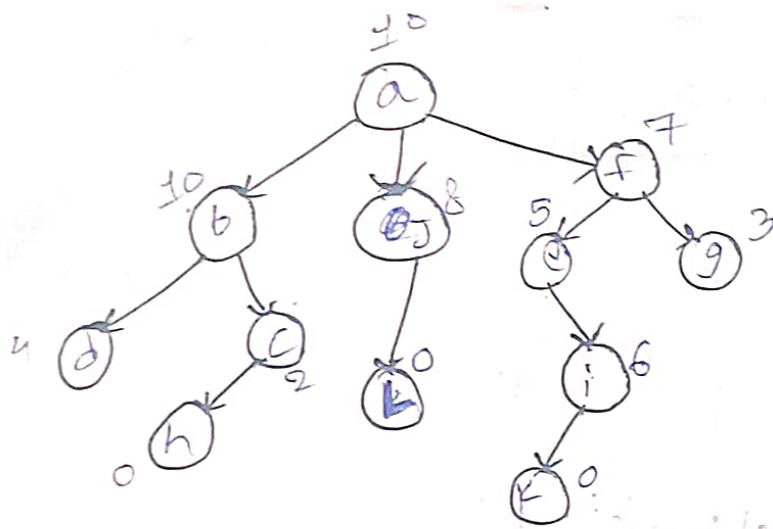
Root Node



* This is an algorithm that simultaneously searches forward from the initial state & backward from the goal state.

HILL CLIMBING

- * This is a greedy local search algorithm.
- * It moves to the neighbor with the best heuristic value.
- * It doesn't backtrack, once it moves, it forgets previous states.
- * It gets stucked if no neighbor has a better heuristic.



Step 1:-

- * Starts from 'a'
- * To node (b) $\Rightarrow f(b) = h(b) = 10$
- * To node (j) $\Rightarrow f(j) = h(j) = 8$
- * To node (f) $\Rightarrow f(f) = h(f) = 7$ ✓
- * To node (f) $\Rightarrow$ Dead end.

$a \rightarrow f \rightarrow g \Rightarrow$ Dead end.
* As the goal node is not found in this path and there is no backtracking we have to choose another path.

Step 15:

* To node (

This

Variants that Allow Behaviour.

1) Random-restart Hill Climbing

When stuck, it restarts from a new random state.

2) Stochastic Hill Climbing

Chooses randomly among better neighbors.

3) Simulated Annealing

Sometimes chooses worse moves to escape local minima.

4) Beam Search

Keeps multiple paths in memory & chooses the best among them.

Classic Hill climbing

$a[10] \rightarrow f[7] \rightarrow g[3]$

Dead-end occurs.

Modified Hill Climbing (with restart or memory)

$a[10] \rightarrow j[8] \rightarrow k[0]$ (goal reached).

$$\frac{8}{12} = \frac{2}{3} \times \frac{300}{1000} + \frac{11.6}{1000}$$

Constraint Satisfaction Problem (CSP)

A CSP is a problem defined by:

- Variables: $X = (X_1, X_2, \dots, X_n)$
- Domains: Each variable X_i has a set of possible values D_i .
- Constraints: Rules that restrict values of variables.
- Goal: Assign values to variables such that all constraints are satisfied.

Ex:-

- * Map Coloring
- * Sudoku
- * Scheduling Problems.

Solving CSPs:-

- ① Backtracking Search
- ② Forward Checking

(12/08/23)

Search in ROBOTICS

- Perception & Navigation
- Object Detection & Manipulation.
- Human - Robot Interaction (HRI).
- Decision-Making & Learning.

Search in LOGISTICS

- Route Optimization & Last-mile Planning
- Demand Forecasting & Inventory Management
- Warehouse Automation.
- Predictive Maintenance.

- Supply Chain Visibility & Transparency
- Customer Service
- Fraud Detection.

Games

- Enhanced Gameplay & NPC's:
 - Non-player characters
- Procedural Content Generation
- Adaptive Difficulty & Personalization.
- Improved Realism & Immersion.
- Game Development & Testing
- Search Algorithms in Game playing
- Minimax Algorithm
- Adversarial Search.

EXPERT SYSTEMS

- Medical Domain
- Monitoring & Process Control
- Knowledge Domain & Trouble shooting.
- Finance
- Manufacturing
- Customer Support.

Variable Ordering

* In CSPs, variable ordering is the strategy of deciding which variable to assign first when solving the problem.

→ Reduces search time by making better decisions early.

Common Heuristics

→ Minimum remaining values (mrV)

→ Degree Heuristic

→ Least constraining value (LCV)

Game Playing (Minimax Algorithm)

→ Used in two-player turn based games like chess, Tic-Tac-Toe, checkers.

→ MAX $\Rightarrow$ maximises the score

MIN $\Rightarrow$ minimises the score.

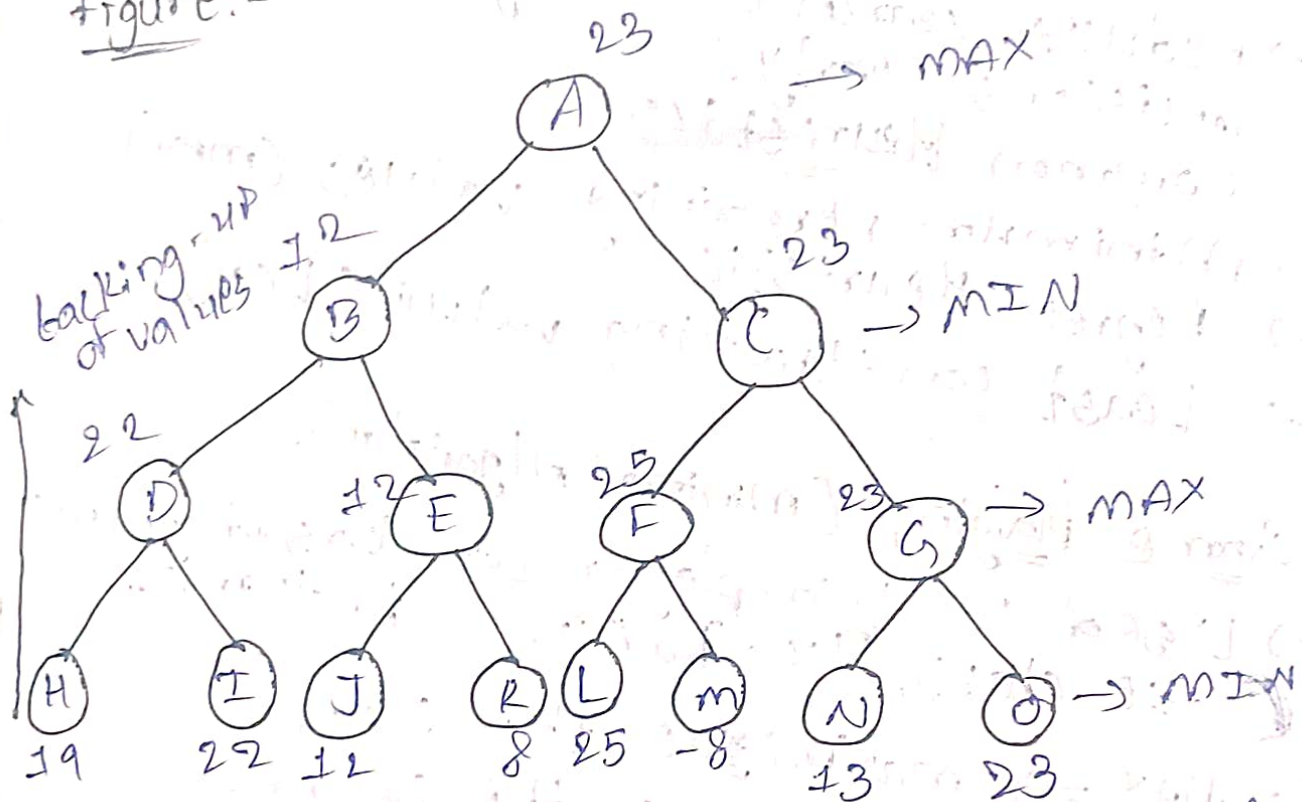
* The algorithm explores all possible moves until a terminal state (win/loss/draw) & back-propagates scores to decide the optimal move.

Steps:-

- 1) Generate the game tree up to a depth.
- 2) Evaluate terminal states with a heuristic value.
- 3) MAX chooses the highest value; MIN chooses the lowest.
- 4) Back up these values to decide the root's best move.

Ex:- In Tic-Tac-Toe if MAX can win in 2 moves, it chooses that particular path even though MIN can block it.

Figure:-



Game start - Player 1 begins the game.

Game tree depth - The algorithm builds a 4-level game tree showing possible moves.

Leaf values - H, I, J, K, L, M, N, O gets their scores from STATIC EVALUATION function.

Level 3 (Max level) - Each node picks up the max value from its child node.

Level 2 (min level) - Each node here picks the min value from its child node.

Value at root (A) - After backing up value from the tree, node 'A' gets the value 23.

Alpha-Beta Pruning (Optimized ~~Pruning~~ Minimax)

→ This is a search optimization for minimax that skips branches that won't affect the final decision.

* This reduces the no of nodes, makes search faster.

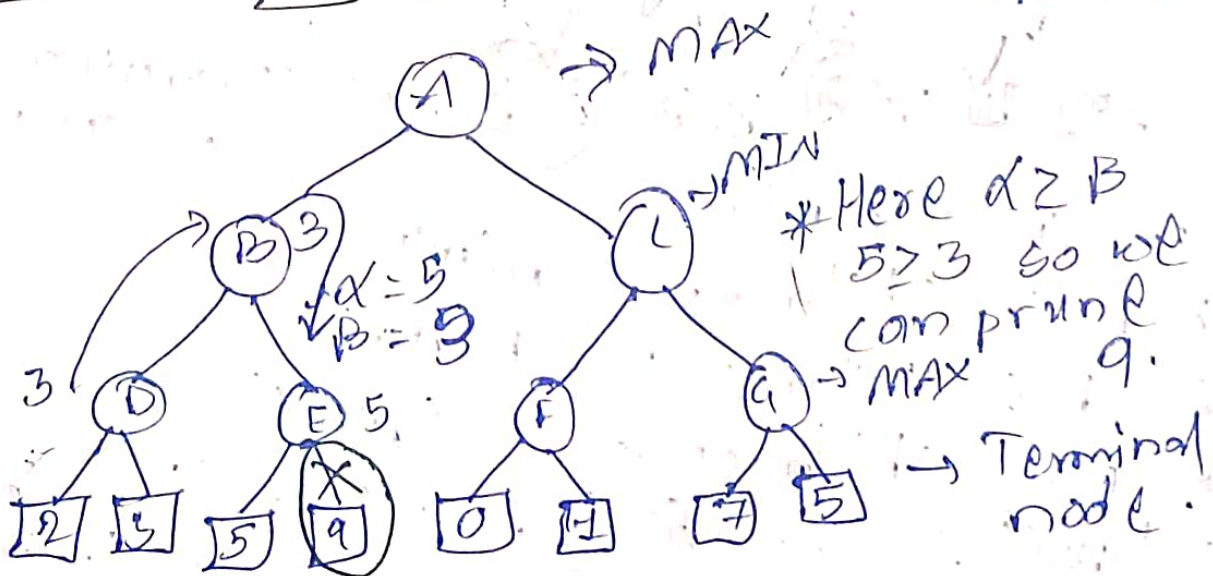
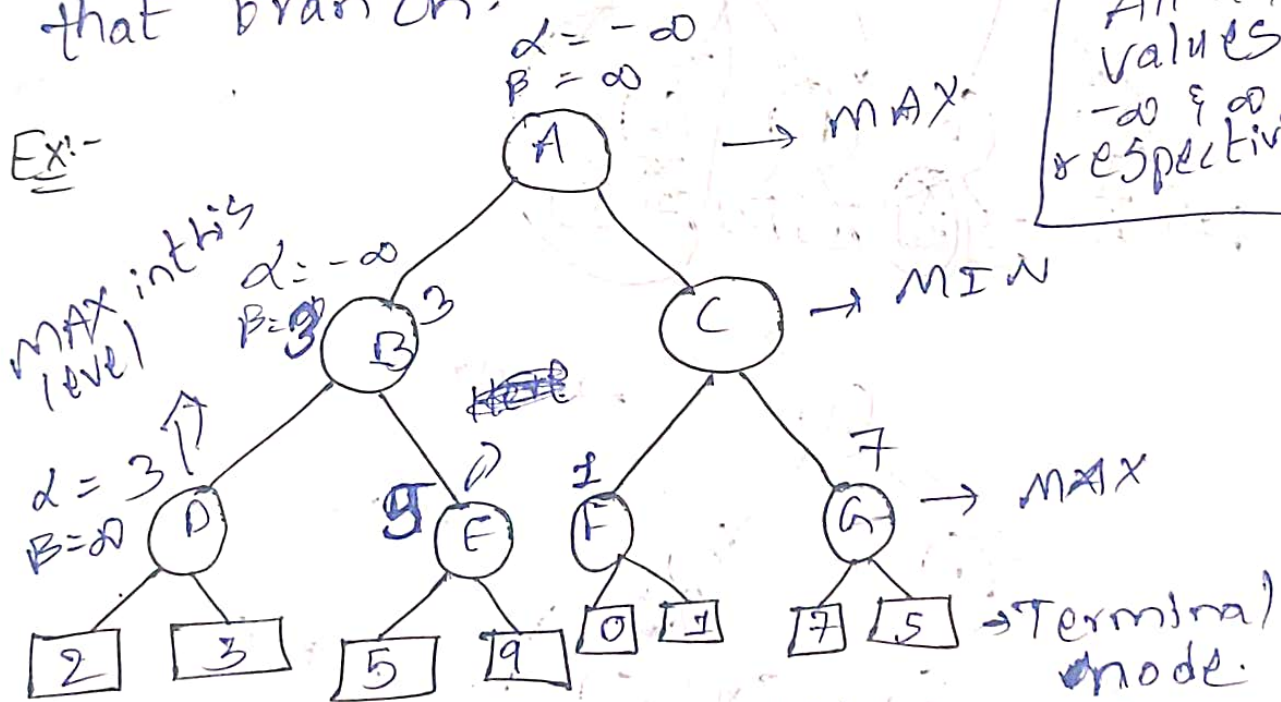
Key Idea:

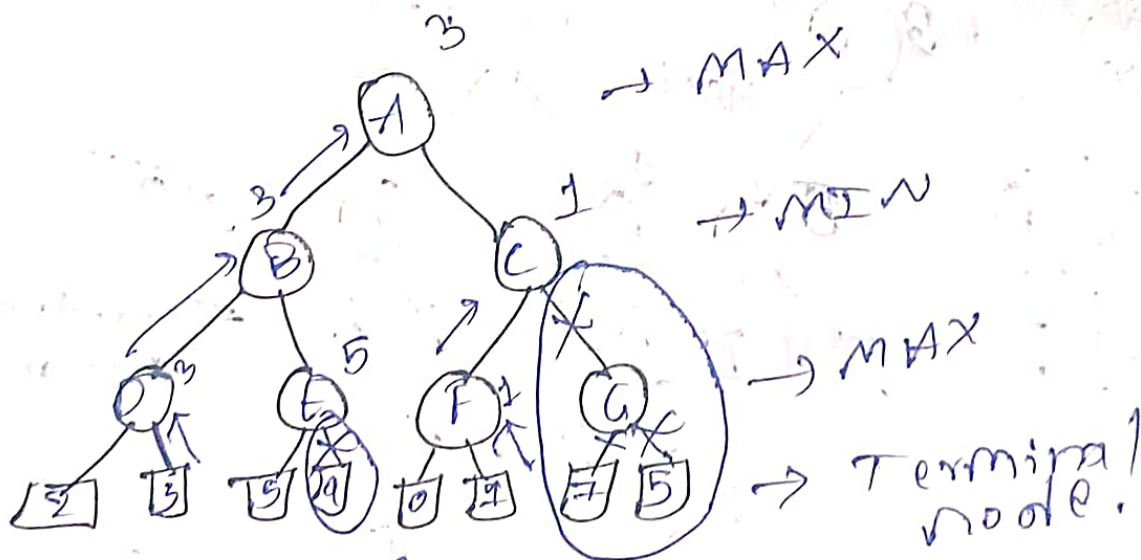
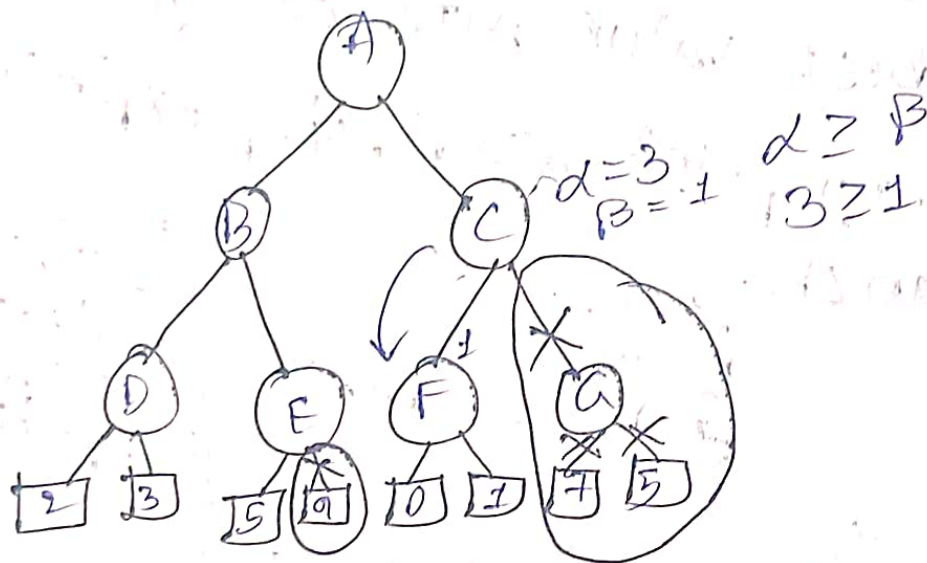
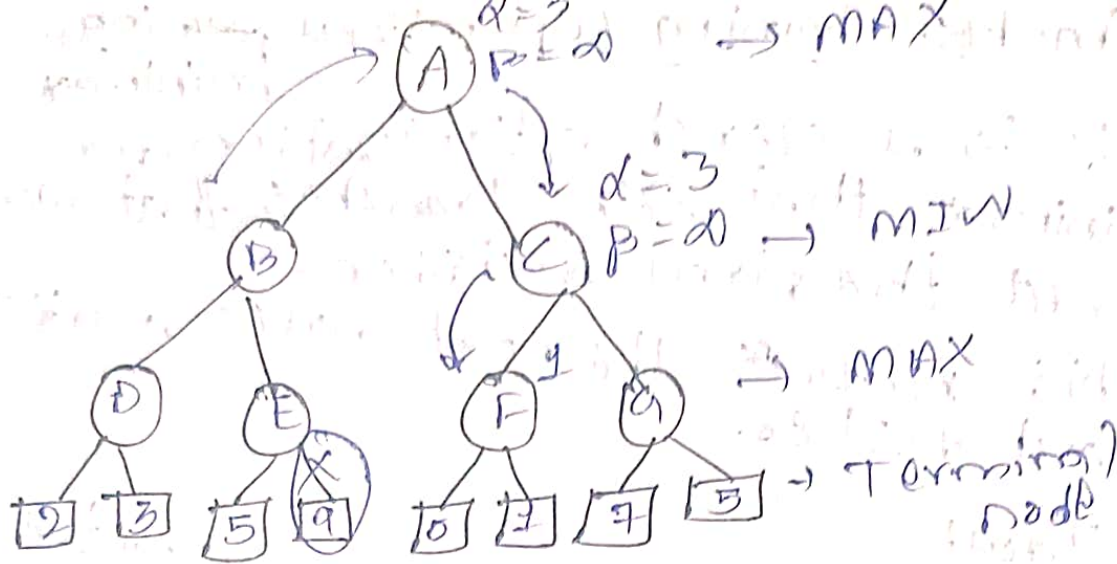
α → Best value MAX can guarantee

β → Best value MIN can guarantee

* If at any point $\alpha \geq \beta$, stop exploring that branch.

Ex:-





- * A = root (MAX)
- * B, C = children of A (MIN)
- * D, E = children of B (MAX)
- * F, G = children of C (MIN)
- * Leaves = 2, 3, 5, 9, 0, 1, 7, 5 (terminal values)

$\alpha = \text{best (highest) for MAX so far;}$

$\beta = \text{best (lowest) for MIN so far;}$